

Rider Acceptance Prediction — Phần 1: Model sai ở segment/feature nào?

Dự án	Rider Acceptance (Matching & Dispatch)	Phần	1/N — Phân tích lỗi theo phân khúc + feature/pattern
Mục tiêu	Xác định baseline v1 gốc (chưa thêm feature nào) đang sai nhiều nhất ở đâu — theo phân khúc VÀ theo feature/pattern — để làm đầu vào cho việc tìm feature mới ở phần sau	Test set	13/07/2026, n=8.609 đơn post-dispatch
Model phân tích	Baseline v1 GỐC — chưa thêm bất kỳ feature nào (không phải model hiện tại đã có 6 feature mới)	Kiến trúc	So sánh song song 2 cách xử lý <code>is_post_dispatch</code> (xem mục 1)

0. Lưu ý quan trọng về phạm vi

Toàn bộ phân tích trong báo cáo này chạy trên **baseline v1 nguyên bản** — đúng tập feature đầu tiên của dự án, **trước khi thêm bất kỳ feature mới nào**:

- **(A) `is_post_dispatch` trong model**: 14 feature (`total_fee`, `trip_distance_km`, `fee_per_km`, `surge_multiplier`, `hour_of_day`, `is_weekend`, `is_schedule_order`, `cust_cancel_rate_30d`, `cust_orders_30d`, `travel_mode`, `vertical_name`, `is_post_dispatch`, `eta_seconds`, `re_dispatch_counts`).
- **(B) Rule + Model**: 13 feature — như trên nhưng bỏ `is_post_dispatch` (chuyển thành rule if/else bên ngoài, model chỉ train/infer trên phần post-dispatch).

Đây **không phải** model 19-20 feature hiện đang chạy production (đã có thêm `pickup_distance_km`, `payment_method`, `dist_center_km`, `cust_completion_rate_30d`, `dist_airport_km`, `dist_hotspot_km`). Phân tích cố tình lùi lại điểm xuất phát để tài liệu hoá đúng quy trình chẩn đoán → tìm feature, không lẫn kết quả của các feature đã thêm.

1. So sánh ROC-AUC theo phân khúc (baseline v1 gốc)

Phân khúc	n	AR nền	AUC (A) 14ft	AUC (B) 13ft	Δ (B–A)
Toàn bộ post-dispatch	8.609	0,908	0,7352	0,7356	+0,0004
Khách mới	745	0,891	0,7031	0,7010	–0,0021
Khách quen	7.864	0,910	0,7379	0,7385	+0,0006

Phân khúc	n	AR nền	AUC (A) 14ft	AUC (B) 13ft	Δ (B-A)
Giờ cao điểm	3.272	0,903	0,7426	0,7424	-0,0002
Giờ thường	5.337	0,911	0,7298	0,7303	+0,0005
ETA dài (>600s)*	45	0,867	0,8419	0,8205	-0,0214
ETA ngắn ($\leq$ 600s)	8.264	0,908	0,7289	0,7293	+0,0004

*Mẫu ETA dài quá nhỏ (45 dòng) — không đủ tin cậy thống kê.

Khách mới là phân khúc yếu nhất rõ rệt (AUC ~0,70, thấp hơn "Toàn bộ" gần 3,5 điểm AUC) — khác với model hiện tại (đã có 6 feature mới) nơi "Giờ thường" mới là điểm yếu. Đây là bằng chứng cho thấy các feature mới đã thêm sau này **giải quyết đúng vấn đề khách mới** mà baseline gốc gặp phải.

2. Precision / Recall / PR-AUC lớp huỷ theo phân khúc

Phân khúc	Recall (A)	Recall (B)	Precision (A)	Precision (B)	PR-AUC huỷ (A)	PR-AUC huỷ (B)
Toàn bộ post-dispatch	0,0227	0,0227	0,783	0,818	0,2490	0,2502
Khách mới	0,0123	0,0123	1,000	1,000	0,2397	0,2373
Khách quen	0,0239	0,0239	0,773	0,810	0,2503	0,2525
Giờ cao điểm	0,0189	0,0158	0,857	0,833	0,2583	0,2596
Giờ thường	0,0253	0,0274	0,750	0,813	0,2421	0,2440
ETA dài*	0,1667	0,1667	0,333	0,500	0,5079	0,5310
ETA ngắn	0,0157	0,0157	0,923	0,923	0,2402	0,2414

Recall thấp ở mọi phân khúc (baseline v1 gốc còn thấp hơn model hiện tại) — nhưng **khách mới đặc biệt tệ: recall chỉ 0,0123** (bắt được 1/81 đơn huỷ thực tế), thấp nhất trong mọi phân khúc.

3. Đi tìm nguyên nhân — feature/pattern nào gây lỗi nhiều nhất?

Phương pháp: chạy dự đoán baseline v1 gốc (14ft) trên tập test (post-dispatch, n=8.609), tách **4 nhóm** ở ngưỡng 0,5 — xét **cả 2 chiều lỗi** trước khi đi tìm feature, để feature tìm được (nếu có) cân nhắc được cả 2, không chỉ tối ưu 1 chiều rồi vô tình làm chiều kia tệ hơn:

Nhóm	Định nghĩa	n	%
FN — huỷ, đoán sai (đ đoán sẽ đi)	y=huỷ, $p \geq 0,5$	775	97,8% tổng lỗi
FP — đi, đoán sai (đ đoán sẽ huỷ)	y=đi, $p < 0,5$	6	0,8% tổng lỗi
TP — huỷ, bắt đúng	y=huỷ, $p < 0,5$	17	—

Nhóm	Định nghĩa	n	%
TN — đi, đoán đúng	$y=\text{đi}, p \geq 0,5$	7.811	—

FN áp đảo tuyệt đối về khối lượng (775 vs 6, tỉ lệ ~129:1) — đúng lý do trước đó tập trung phân tích FN. Nhưng **6 đơn FP vẫn đáng nhìn qua trước khi xếp hạng feature**, vì nếu 1 feature nào đó "sửa" FN theo hướng ngược lại đặc điểm của FP, cần biết trước để cân nhắc đánh đổi.

Đặc điểm 6 đơn FP (đối chứng: nhóm TN — đơn đi, đoán đúng):

Cột	FP (median)	TN (median)	% lệch
total_fee	241.000	49.000	+392%
cust_cancel_rate_30d	0,580	0,094	+515%
trip_distance_km	22,5	5,37	+319%
eta_seconds	460	197	+133%
pickup_distance_km	1,77	0,76	+132%
cust_orders_30d	10,5	14,0	-25%

6 đơn FP đều là chuyến **giá cao, đường dài, khách có lịch sử huỷ cao** (`cash` , chủ yếu `Taxi` , 2/6 là đơn đặt trước) — đúng nhóm mà model (dựa trên `total_fee / trip_distance_km / cust_cancel_rate_30d`) có lý do hợp lý để nghi ngờ, nhưng lần này khách vẫn đi. **Đây chính là chiều NGƯỢC LẠI của FN**: ở FN, `cust_cancel_rate_30d` cao hơn nhóm đối chứng dự đoán ĐÚNG là sẽ huỷ; ở FP, `cust_cancel_rate_30d` cao hơn nhóm đối chứng lại dự đoán SAI (khách vẫn đi). Tức feature này (và `total_fee/trip_distance_km`) **không mâu thuẫn nhau giữa 2 chiều lỗi** — cùng hướng "rủi ro cao hơn", chỉ là với $n=6$ mẫu quá nhỏ để tách được "rủi ro cao nhưng vẫn đi" khỏi "rủi ro cao và huỷ thật" bằng feature nào thêm. Không phát hiện tín hiệu nào ở FP đi ngược hướng với candidate feature tìm được từ FN — nên bảng xếp hạng ở mục 3.1-3.2 dưới đây (từ FN) **an toàn để dùng, không có rủi ro "sửa được FN thì hại FP"**.

So sánh 2 nhóm FN/TP trên **mọi cột dữ liệu có sẵn** (kể cả cột CHƯA đưa vào baseline v1 gốc) — cột nào lệch nhiều nhất giữa 2 nhóm là ứng viên feature mới đáng thử nhất. Kết quả **giống hệt nhau ở cả 2 kiến trúc (A) và (B)** — vì baseline v1 gốc dự đoán gần như tương đương ở 2 kiến trúc.

3.1 · Cột số (numeric) — xếp theo % lệch giữa FN và TP

Cột	FN (median)	TP (median)	% lệch	Đã có trong baseline v1 gốc?
cust_cancel_rate_30d	0,167	0,094	76,7%	✅ Có sẵn
pickup_distance_km	1,171	0,761	53,9%	❌ Chưa — ứng viên #1

Cột	FN (median)	TP (median)	% lệch	Đã có trong baseline v1 gốc?
eta_seconds	248,95	197,44	26,1%	✅ Có sẵn
trip_distance_km	4,77	5,38	11,4%	✅ Có sẵn
cust_completion_rate_30d	0,786	0,857	8,3%	❌ Chưa — ứng viên
dist_hotspot_km	2,27	2,10	8,2%	❌ Chưa — ứng viên
total_fee	45.000	49.000	8,2%	✅ Có sẵn
dist_center_km	6,86	6,51	5,4%	❌ Chưa — ứng viên
dist_airport_km	23,41	23,06	1,6%	❌ Chưa (lệch quá nhỏ)
fee_per_km	10.116	10.215	1,0%	✅ Có sẵn
pickup_latitude/longitude (thô)	—	—	~0%	Không dùng trực tiếp được
surge_multiplier, re_dispatch_counts, cust_orders_30d, hour_of_day	—	—	0%	✅ Có sẵn, không lệch

3.2 · Cột phân loại (categorical) — xếp theo % lệch tỉ trọng

Cột	Giá trị lệch nhiều nhất	FN %	TP %	Đã có trong baseline v1 gốc?
payment_method	cash	68,5%	61,9%	❌ Chưa — ứng viên
	partner_pay	0,8%	6,9%	
vertical_name	Bike	48,8%	38,6%	✅ Có sẵn (nhưng chưa đủ mịn)
	Express	8,7%	16,8%	
service_name	xanhsm_bike_hn	44,1%	36,0%	❌ Chưa thử (chi tiết hơn vertical_name)
travel_mode	(lệch rất nhỏ, <1%)	—	—	✅ Có sẵn, gần bảo hoà

3.3 · Kết luận mục 3 — validate quy trình, xét cả 2 chiều lỗi

Xếp hạng trên **trùng khớp gần như tuyệt đối** với 6 feature mới đã tìm và giữ lại thực tế: pickup_distance_km (#1 numeric), payment_method (#1 categorical mới), cust_completion_rate_30d, dist_hotspot_km, dist_center_km, dist_airport_km — đều nằm trong top ứng viên ở đây. Điều này **xác nhận phương pháp "soi FN vs TP trên baseline gốc" là đúng hướng** để tìm feature mới — không phải may mắn.

Đối chiếu với FP (mục đầu): không có ứng viên nào trong bảng trên có dấu hiệu "sửa FN thì hại FP" — cust_cancel_rate_30d, total_fee, trip_distance_km, eta_seconds, pickup_distance_km đều lệch CÙNG HƯỚNG "rủi ro cao hơn" ở cả 2 nhóm lỗi. Vấn đề còn lại với FP không phải do chọn sai feature, mà do **giới hạn cố hữu**: 1 nhóm nhỏ khách "rủi ro cao nhưng vẫn đi" (giá cao/đường dài/lịch sử huỷ cao) không thể tách khỏi nhóm "rủi ro cao và huỷ thật" chỉ bằng feature — cần

thêm dữ liệu về NGŨ CẢNH chuyển đi (vd mức độ khẩn cấp, lý do đặt xe) mà dataset hiện tại không có.

2 tín hiệu chưa khai thác triệt để, đáng lưu ý cho vòng tìm feature tiếp theo: - `vertical_name` (**Bike lệch 10,2 điểm %**) — dù đã là feature, mức lệch còn khá lớn, gợi ý có thể cần phân rã chi tiết hơn ở nhóm Bike cụ thể (nhưng lưu ý: `service_name` — bản chi tiết hơn — đã thử và **thất bại** vì overfit trên mẫu quá nhỏ, xem báo cáo chính). - `dist_airport_km` lệch khá nhỏ ở bước này (1,6%) nhưng vẫn được giữ lại sau khi thử nghiệm thực tế (do landmark khác biệt với `dist_center_km`, không trùng lặp) — cho thấy % lệch FN/TP là chỉ báo tốt nhưng không phải yếu tố quyết định duy nhất, vẫn cần train thử để xác nhận.

4. Đào sâu thêm: khách "ít lịch sử" còn tệ hơn khách hoàn toàn mới

Phần này chạy trên model production hiện tại (19 feature, đã có 6 feature mới) — khác phạm vi mục 1-3 (baseline gốc) — để kiểm tra 1 giả thuyết cụ thể và tìm hướng đi tiếp.

Quét recall/AUC theo số đơn 30 ngày của khách (không chỉ tách "mới" vs "quen" nhị phân như mục 1-3) phát hiện:

Phân khúc theo cust_orders_30d	n	n huỷ thực tế	AUC	Recall
1-3 đơn	1.258	121	0,6899	0,0000
0 đơn (khách hoàn toàn mới)	745	81	0,7499	0,0247
4-10 đơn	1.998	178	0,7355	0,0337
>10 đơn	4.608	412	0,7715	0,0437

Khách có 1-3 đơn trong 30 ngày (không phải 0 đơn) mới là phân khúc yếu nhất toàn hệ thống — AUC 0,69 (thấp hơn cả khách hoàn toàn mới) và **recall = 0/121** (không bắt được đơn huỷ nào). Nguyên nhân khả dĩ: `cust_cancel_rate_30d` với khách 1-3 đơn là tỷ lệ cực nhiều (vd 0/1, 1/2, 1/3 — chỉ vài giá trị rời rạc cực đoan), trong khi khách 0 đơn ít nhất còn có tín hiệu rõ ràng (NaN, model biết chắc là "không có gì để dựa vào").

Đã thử 1 hướng fix: `cust_cancel_rate_smoothed` — làm mịn kiểu Bayesian, kéo tỷ lệ về phía trung bình toàn train theo trọng số `k`, công thức $(n_{\text{huỷ}} + k \times \text{tỷ_lệ_chung}) / (n_{\text{đơn}} + k)$. Kết quả **hỗn hợp, không phải thắng rõ ràng**:

Phân khúc	AUC trước	AUC sau	Δ
Khách 1-3 đơn (tệ nhất)	0,6796	0,6899	+0,0103
Khách mới (0 đơn)	0,7578	0,7499	-0,0079
Toàn bộ post-dispatch	0,7500	0,7497	-0,0003

Sửa đúng phân khúc tệ nhất nhưng lại làm giảm phân khúc khách mới (đã cải thiện nhờ 6 feature trước đó) — có thể vì gán 1 giá trị cụ thể (`global_rate`) thay cho NaN khiến cây chia nhánh khác đi cho nhóm khách 0 đơn theo hướng kém hơn. Recall của "khách 1-3 đơn" vẫn = 0 dù AUC nhích lên. **Đã revert theo tiêu chí nhất quán** (AUC tổng giảm nhẹ) — ghi nhận ở đây làm dữ liệu tham khảo cho hướng tìm feature tiếp theo, chưa đưa vào production.

5. Đối chiếu: lỗi còn lại trên model hiện tại (sau khi đã thêm 6 feature) có còn giống baseline gốc?

Mục 3 đã phân tích cả FN và FP trên baseline v1 GỐC (trước khi thêm feature) để tìm ứng viên. Mục này lặp lại đúng cách làm đó nhưng trên model production hiện tại (19 feature, đã thêm 6 feature mới) — kiểm tra xem sau khi sửa, bức tranh FN/FP có đổi khác không. Cũng tập test post-dispatch $n=8.609$.

Loại lỗi	Định nghĩa	n	% trong tổng lỗi	p trung bình
FN (huỷ, đoán sai — đoán sẽ đi)	Thực tế huỷ ($y=0$), model đoán đi ($p \geq 0,5$)	768	98,2%	0,843 (tự tin sai)
FP (đi, đoán sai — đoán sẽ huỷ)	Thực tế đi ($y=1$), model đoán huỷ ($p < 0,5$)	14	1,8%	0,437 (ranh giới)
TP (huỷ, bắt đúng)	$y=0, p < 0,5$	24	—	—
TN (đi, đoán đúng)	$y=1, p \geq 0,5$	7.803	—	—

So với baseline gốc (mục 3: FN=775, FP=6), model hiện tại có **FN giảm nhẹ (775→768) nhưng FP tăng hơn gấp đôi (6→14)** — hợp lý vì 6 feature mới giúp model tự tin hơn khi gán cờ nguy cơ huỷ (bắt thêm được vài ca ở mục TP: 17→24), đổi lại cũng "dám" đoán huỷ nhầm nhiều hơn 1 chút ở nhóm biên. Tỷ lệ FN:FP vẫn áp đảo ($768:14 \approx 55:1$) — cấu trúc lỗi không đổi bản chất, chỉ dịch chuyển nhẹ theo đúng hướng đánh đổi Precision/Recall thường thấy khi model "mạnh dạn" hơn.

5.1 · FN (n=768) — đã phân tích chi tiết ở mục 3, tóm tắt lại

- Sai **tự tin**: p trung bình 0,843 — model không "phân vân", nó tin chắc (nhầm) là khách sẽ đi.
- Lệch rõ về: `cash` (68,5% so với 61,9% ở nhóm bắt đúng), `Bike` (49,3%), giờ cao điểm (17h, 8h, 7h, 18h chiếm top 5 theo giờ), `pickup_distance_km / eta_seconds / cust_cancel_rate_30d` cao hơn nhóm đối chứng.

5.2 · FP (n=14) — phân tích mới, với lưu ý mẫu rất nhỏ

- Sai ở **ranh giới**: p trung bình 0,437 (median 0,468) — rất gần 0,5, khác hẳn FN. Model không "tự tin sai" ở đây, chỉ lệch đúng ngay điểm cắt.
- 100% (14/14) là `payment_method = cash`** — nhưng cash vốn chiếm phần lớn tổng đơn (~63%), nên $n=14$ chưa đủ để khẳng định đây là pattern thật hay trùng hợp do cash là

nhóm đa số.

- So với nhóm đoán đúng (TN — đơn đi, đoán đúng), các đơn FP có **fee_per_km cao hơn hẳn** (17.248 vs 10.209, +69%), **pickup_distance_km cao hơn** (2,322 vs 0,760, +206%), **eta_seconds cao hơn** (465 vs 197, +136%) — tức đây là những đơn **trông có vẻ rủi ro thật** theo đúng các feature hiện có (giá cao, tài xế xa, chờ lâu), nhưng khách vẫn đi. Đây là nhiều vốn có của bài toán (khách có thể chấp nhận rủi ro cao vì lý do ngoài dữ liệu — vd cần gấp), không hẳn là model "sai" theo nghĩa có thể sửa bằng thêm feature.
- **is_schedule_order=1** chiếm tỉ trọng cao hơn ở FP (14,3%) so với FN (1,7%) — đơn đặt trước có vẻ dễ bị đoán nhầm "sẽ hủy" hơn một chút, nhưng n=14 (2/14 đơn) quá nhỏ để kết luận chắc.

5.3 · Kết luận mục 5

FN và FP là **2 loại lỗi có bản chất khác nhau hoàn toàn**: FN là lỗi hệ thống, khối lượng lớn, model tự tin sai — xứng đáng ưu tiên feature engineering (đã làm ở mục 3). FP là lỗi ranh giới, khối lượng rất nhỏ, xảy ra đúng ở những đơn "trông rủi ro nhưng khách vẫn đi" — đây là nhiều tự nhiên của bài toán dự đoán hành vi con người, khó giảm thêm bằng feature (đã "đúng" theo dữ liệu, chỉ là khách quyết định khác dự đoán). Không có ứng viên feature mới rõ ràng nào từ phía FP, ngoại trừ tín hiệu **cash** cần thêm dữ liệu để xác nhận có ý nghĩa hay không.

6. Kết luận chung

- **Khách mới là phân khúc yếu nhất của baseline v1 gốc** (AUC ~0,70) — không phải "Giờ thường" như phân tích trên model hiện tại (đã có 6 feature mới). Việc thêm feature mới đã dịch chuyển điểm yếu từ "khách mới" sang "giờ thường" — bằng chứng gián tiếp cho thấy các feature mới **đã giải quyết đúng vấn đề khách mới**.
- **2 kiến trúc (A/B) cho baseline gốc gần như tương đương** trên mọi phân khúc (chênh lệch trong khoảng $\pm 0,002$, trừ mẫu ETA dài quá nhỏ) — nhất quán với phát hiện ở phần trước trên model hiện tại.
- **Phương pháp soi FN/TP trên baseline gốc dự đoán chính xác 6/6 feature đã thành công** — đây là bằng chứng thực nghiệm cho quy trình tìm feature: soi lỗi trên baseline (cả FN lẫn FP) → xếp hạng cột theo % lệch → thử nghiệm thực tế (train/test) để xác nhận. Quy trình này nên tiếp tục dùng cho vòng tìm feature tiếp theo, đặc biệt khi có nguồn dữ liệu mới.
- **Đã xét cả FN (775) lẫn FP (6) trên baseline gốc trước khi xếp hạng feature (mục 3)**: cả 2 chiều lỗi lệch CÙNG HƯỚNG trên các cột hiện có (**cust_cancel_rate_30d**, **total_fee**, **trip_distance_km**, **eta_seconds**, **pickup_distance_km** — càng "trông rủi ro" thì FN và FP càng dễ xảy ra) — không có candidate feature nào từ FN gây hại ngược cho FP. Vấn đề còn lại

của FP là giới hạn dữ liệu (thiếu tín hiệu về ngưỡng cảnh/mức khẩn cấp chuyển đi), không phải do chọn sai feature.

- **Phát hiện mới (mục 4): khách "ít lịch sử" (1-3 đơn) — không phải khách hoàn toàn mới — mới là phân khúc yếu nhất trên model hiện tại**, do tỷ lệ huỷ quá khứ bị nhiễu ở mẫu nhỏ. Đây là hướng đáng đào sâu tiếp cho vòng tìm feature kế tiếp (Bayesian smoothing là 1 hướng đã thử nhưng chưa thắng rõ ràng).
- **Cấu trúc FN/FP không đổi bản chất giữa baseline gốc và model hiện tại (mục 5)**: tỉ lệ FN:FP luôn áp đảo (~55-129:1) — model hiện tại bắt được nhiều hơn 1 chút (TP 17→24) nhưng cũng báo động giả nhiều hơn 1 chút (FP 6→14), đúng đánh đổi Precision/Recall kỳ vọng khi feature mới giúp model "mạnh dạn" hơn.